

Title: AutoJudge (Predicting Programming Problem Difficulty)

Name: Arvind Verma

En No: 22116019

1. Problem Statement

Online competitive programming platforms such as Codeforces, CodeChef, and Kattis classify programming problems into difficulty categories (Easy, Medium, Hard) and often associate a numerical difficulty score.

Currently, this process relies heavily on human judgment, editorial feedback, and post-contest statistics, which can be subjective, time-consuming, and inconsistent for newly added problems.

The objective of this project is to design an automated system (AutoJudge) that can:

1. Classify a programming problem into:
 - Easy
 - Medium
 - Hard
2. Predict a numerical difficulty score for the problem

The prediction must rely only on the textual description of the problem, without using solution data or user statistics.

2. Dataset Description

The dataset consists of programming problems collected from online judges (e.g., Kattis-style problems).

Each data sample contains the following fields:

- Title – Short name of the problem
- Problem Description – Full problem statement
- Input Description – Input format and constraints
- Output Description – Expected output format
- Problem Class – Ground truth label (Easy, Medium, Hard)

- Problem Score – Numerical difficulty score (continuous value)

Dataset Characteristics

- Stored in JSON Lines (.jsonl) format
- Each line represents one problem
- Contains both categorical (difficulty class) and continuous (difficulty score) targets
- Text length and complexity vary significantly across difficulty levels

3. Methodology and Feature Engineering

The system is designed as a two-stage learning pipeline:

3.1 Text Preprocessing

All text fields are:

- Lowercased
- Cleaned of extra whitespace and HTML-like artifacts
- Combined into a single text input:

3.2 Feature Engineering

Two types of features are extracted:

A. Textual Features (TF-IDF)

- **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorization
- Captures important keywords such as:
 - graph, dp, recursion, bfs, dfs, segment tree
- Helps distinguish algorithmic complexity from plain text

B. Numeric Complexity Features

Hand-crafted numeric signals indicating difficulty:

- Total character count
- Word count
- Count of mathematical symbols (+ - * / < >)
- Presence of large constraints (e.g., 10^5 , 10^9)

These numeric features are **scaled** and concatenated with TF-IDF vectors.

4. Models Used

4.1 Classification Model

- **Task:** Predict difficulty class (Easy / Medium / Hard)
- **Model:** Logistic Regression (baseline) / Random Forest (final)
- **Input:** TF-IDF + numeric features
- **Output:** One of three difficulty classes

4.2 Regression Model

- **Task:** Predict numerical difficulty score
- **Model:** Gradient Boosting Regressor
- **Why Gradient Boosting?**
 - Handles non-linear relationships
 - Robust to mixed feature types
 - Performs better than linear regression on text-derived features

5. Experimental Results

5.1 Classification Performance

Classification Accuracy $\approx$ **82%**

Regression MAE: 7.2

- The model performs well in distinguishing Easy, Medium, and Hard problems based purely on text.
- The regression task is inherently more challenging because:
- Difficulty scores are **subjective**
- Overlap exists between difficulty classes
- Despite this, Gradient Boosting provides reasonable approximations.
-

6. Web Interface

A Streamlit-based web application is built for user interaction.

Features

- Input fields for:
 - Problem Title
 - Problem Description
 - Input Description
 - Output Description
- A Predict button
- Displays:
 - Predicted difficulty class
 - Predicted difficulty score (0–100)
 - Visual progress bar for score interpretation

7. Conclusion

This project demonstrates that programming problem difficulty can be reasonably inferred from textual descriptions alone using NLP and machine learning techniques.

Key Contributions

- Combined TF-IDF and numeric complexity features
- Built a dual-task system for classification and regression
- Successfully deployed a real-time prediction interface using Streamlit

Limitations

- Exact numeric difficulty is hard to infer due to subjective labeling
- Model does not use solution complexity or runtime constraints

